Anexo F —— Guía de Integración Software en el SoC NEORV32.

F.1. Introducción.

El objetivo de este anexo es detallar los mecanismos existentes para compilar, transferir y ejecutar los programas de aplicación de usuario (escritos en lenguaje de alto nivel, C) sobre el hardware del *SoC* NEORV32. Existen dos opciones diferentes para gestionar el contenido de la memoria de instrucciones (`IMEM`):

1. Carga Dinámica (Bootloader): este método se basa en la transferencia del ejecutable binario (`neorv32_exe.bin`) a través del puerto UART. Su principal ventaja reside en la no necesidad de volver a sintetizar el hardware cada vez que se haga un cambio en el software, pero el programa se pierde al reiniciar la placa (memoria volátil).

2. Carga Estática (Imagen de Memoria IMEM): este método se basa en la integración del código compilado en el hardware mediante una memoria ROM. Se genera una imagen VHDL de la memoria (`neorv32_application_image.vhd`) que se integra junto con el resto del diseño en Vivado. Al encender la placa, el *bootloader* del NEORV32 transfiere el contenido de la ROM a la memoria `IMEM` (RAM), de esta forma se consigue que el programa se ejecute al encender la FPGA, pero requiere volver a sintetizar el proyecto ante cualquier cambio en el software.

A continuación, se detallan los pasos a seguir para cada uno de estos procedimientos.

F.2. Carga Dinámica Mediante Bootloader.

El *bootloader* (cargador de arranque) es un pequeño programa de inicio alojado en la memoria ROM del procesador. Las diferentes opciones de este programa se gestionan en la entidad superior del proyecto de Vivado mediante el *generic* `BOOT_MODE_SELECT`.

Para implementar este procedimiento, se necesita que el valor de este *generic* sea `0`, indicando que se usará la ROM interna del *bootloader*. Este programa permite la transferencia de archivos binarios a través de una interfaz serie estándar.

F.2.1. Requisitos Previos.

Antes de realizar la carga de la aplicación de usuario mediante este procedimiento, se necesita haber realizado los siguientes pasos:

1. Haber realizado la creación de un proyecto para el NEORV32 en Vivado, seleccionando la opción de `neorv32_test_setup_bootloader.vhd` como entidad superior del proyecto. Esta configuración activa el modo de *bootloader* necesario para la carga dinámica. Para ello puede consultarse el Anexo E.

2. Programar la placa con el *bitstream* generado en el punto anterior.

3. Compilar el programa de aplicación como ejecutable binario (`neorv32_exe.bin`). Para ello se debe acceder a la carpeta donde se aloje el código fuente y ejecutar el comando `make clean_all exe`:

```
~/neorv32/sw/example/carpeta_ejemplo$ make clean_all exe
Memory utilization:
   text    data     bss     dec     hex filename
   4756       0       0    4756    1294 main.elf
Compiling image generator...
Generating neorv32_exe.bin
Executable size in bytes:
4768
```

4. Disponer de un terminal serie (como Putty, TeraTerm, etc.), en este caso se ha empleado TeraTerm.

F.2.2. Procedimiento.

1. Abrir una nueva sesión en el terminal serie (TeraTerm en este caso).

2. Seleccionar la opción de Puerto Serie (`Serial`) y escoger el puerto donde se encuentre conectada la placa Nexys4 DDR (Fig.1). Pulsar en `OK` para continuar.

3. Para configura el puerto serie correctamente con los parámetros del NEORV32, hacer clic en `Configuración` y seleccionar `Puerto Serial` (Fig.2).

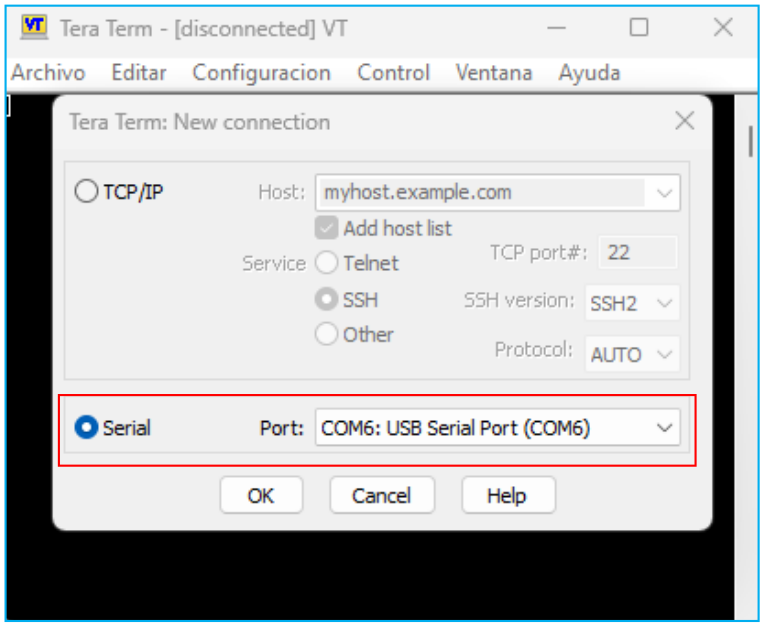

Fig.1. Ventana de nueva conexión en el terminal serie, selección de opción `Serial`.

4. En la ventana emergente Additional settings, seleccionar los siguientes parámetros y pulsar en `OK` (Fig.3):

   - Baud rate: 19200.
   - Data: 8 bit.

- Parity: none.
- Stop: 1 bit.
- Flow control: none.

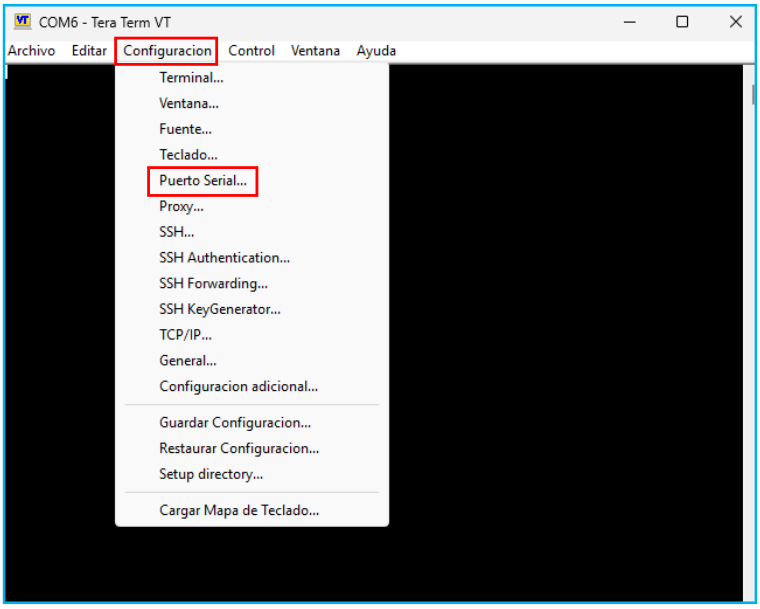

Fig.2. Apertura de la ventana de configuración del puerto serie.

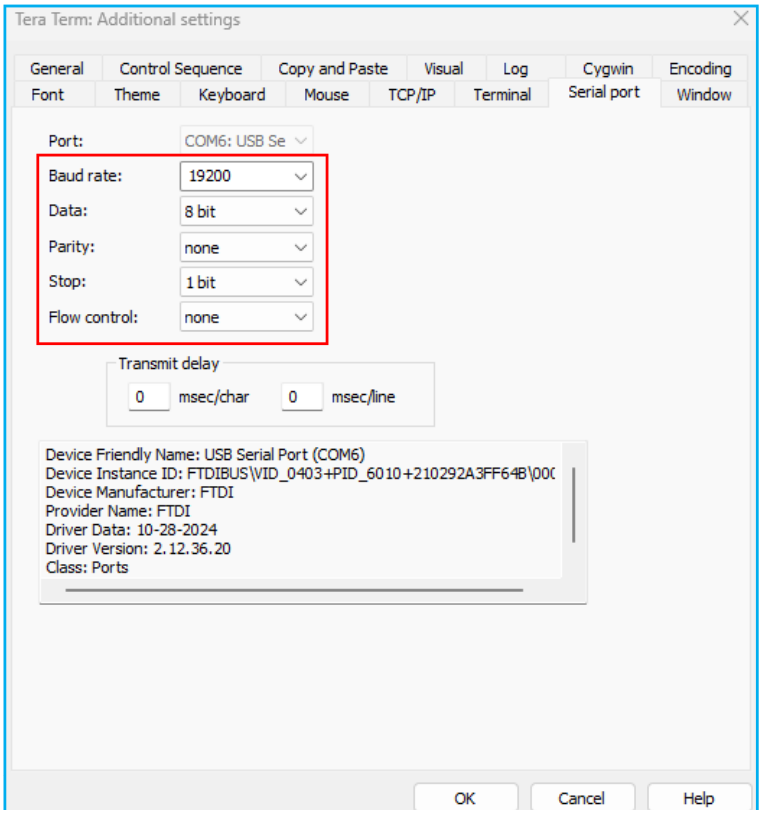

Fig.3. Parámetros de configuración del puerto serie para la conexión con el NEORV32.

5. Una vez configurado el puerto serie, se debe activar el *reset* del NEORV32 (`rstn_i`) mediante el interruptor `SW0` (Fig.4). Tras esto el `LD0` comenzará a parpadear y aparecerá un mensaje del *bootloader* por el terminal (Fig.5). Antes de que transcurran 10 segundos se debe presionar cualquier tecla para acceder al menú del *bootloader*.

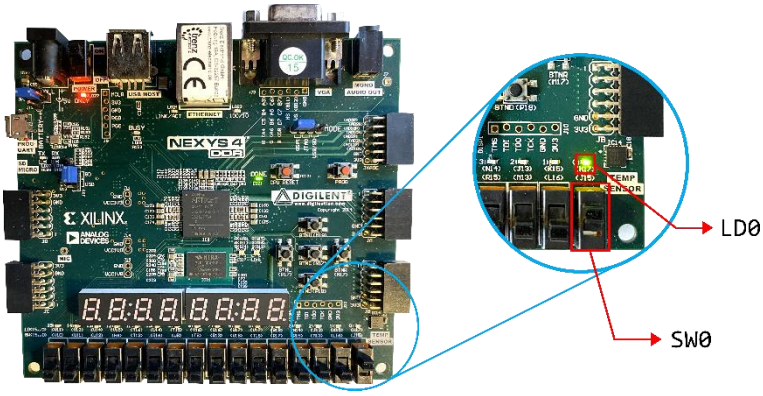

Fig.4. Vista superior del `SW0` y `LD0` de la placa Nexys4 DDR.

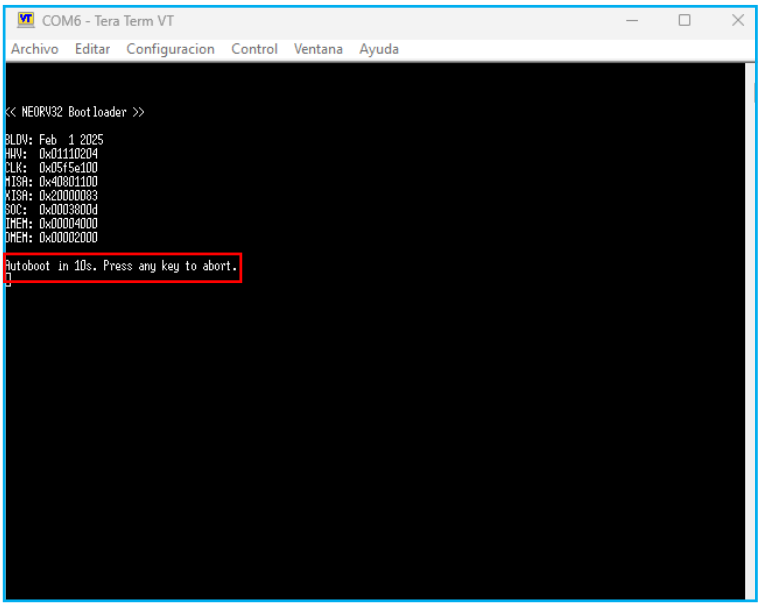

Fig.5. Mensaje de inicio del *bootloader* mostrado por el terminal.

6. Cuando aparezca el menú del *bootloader* en el terminal, se pulsará la tecla '`u`' en el teclado para acceder al modo *Upload* (Fig.6). En este modo, el NEORV32 esperará la transmisión del binario `nerov32_exe.bin` (Fig.7).

7. Para iniciar la transmisión del archivo, pulsamos en `Archivo` y seleccionamos `Enviar Archivo` (Fig.8).

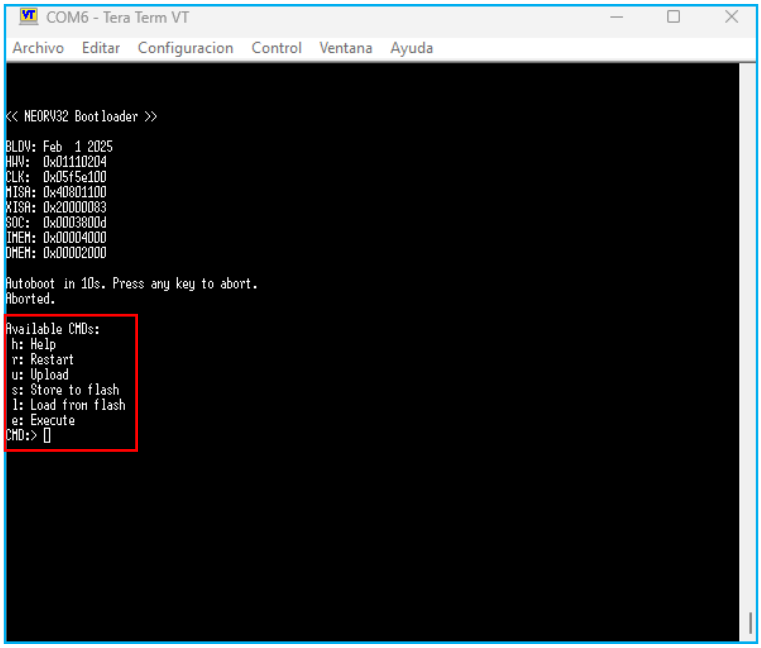

Fig.6. Menú de comandos disponibles en el programa *bootloader*.

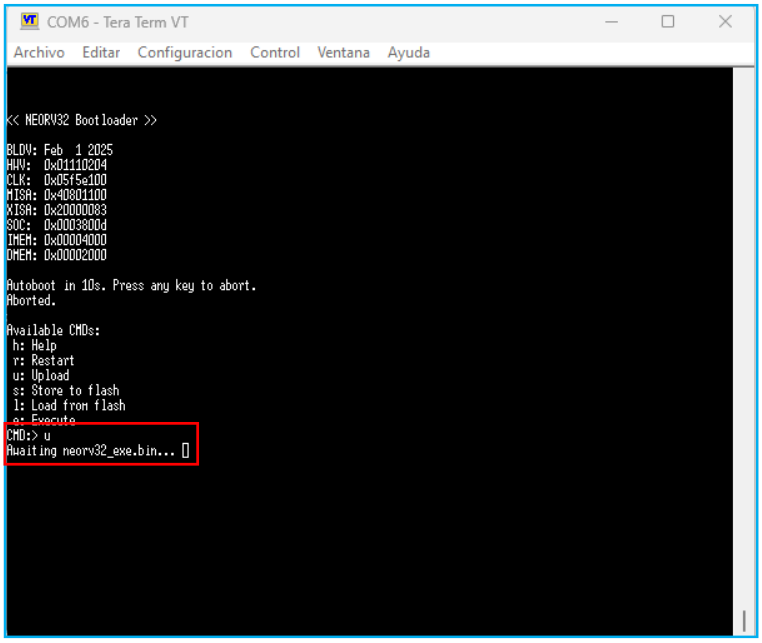

Fig.7. Ejecución del comando *Upload* (‘`u`’).

8. Seleccionamos el archivo `neorv32_exe.bin` correspondiente al programa de aplicación que se quiera ejecutar en el procesador. Además, seleccionamos las opciones `Sequential read` y `Binary` (Fig.9). Pulsamos `OK`.

9. Si la transferencia ha ido bien, en el terminal debería mostrarse el mensaje: “`OK`” (Fig.10).

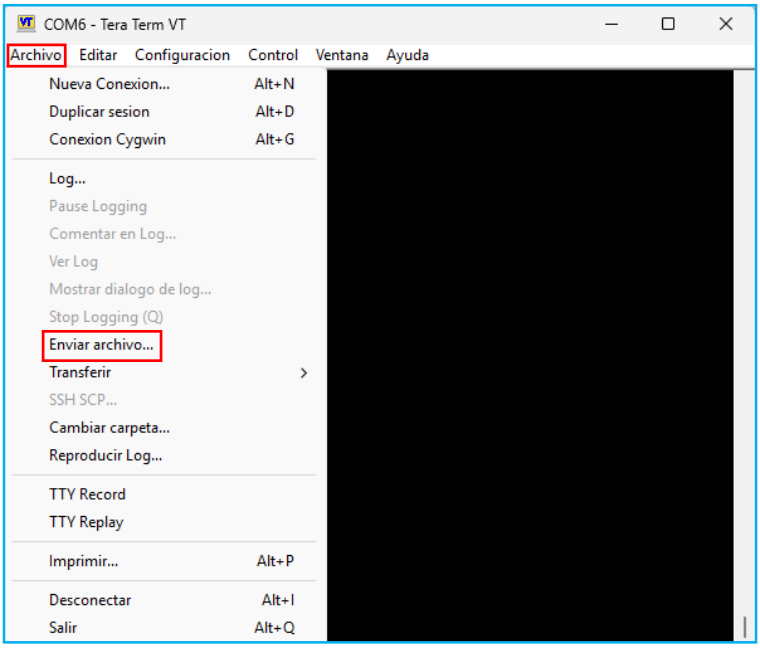

Fig.8. Apertura de ventana para el envío del ejecutable binario.

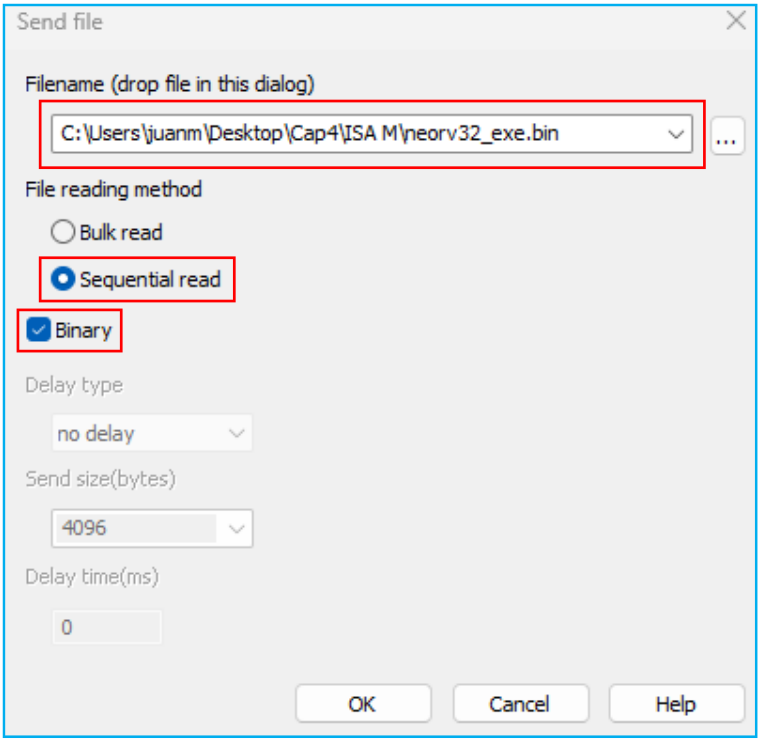

Fig.9. Ventana de envío de archivo por el puerto serie.

10. Si todo fue correcto, para proceder con la ejecución del código de usuario, pulsamos la tecla '`e`' (*Execute*). Como se puede observar en la Fig.11, el programa se ejecuta correctamente.

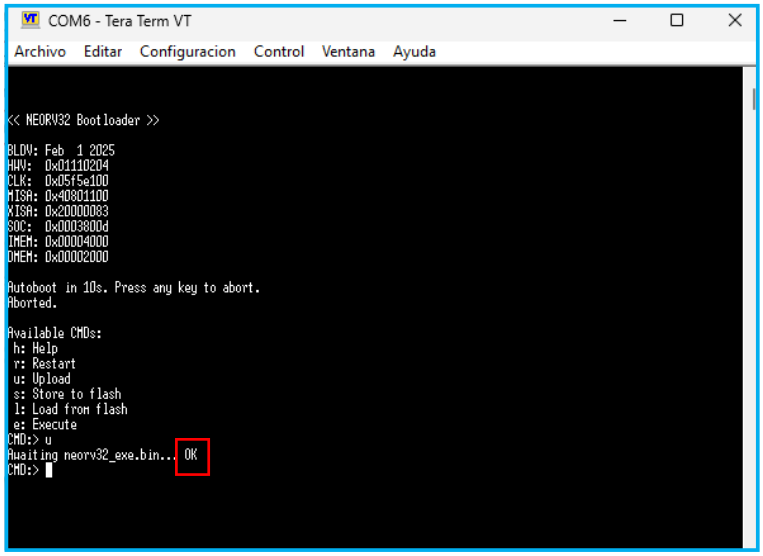

Fig.10. Mensaje de archivo recibido correctamente.

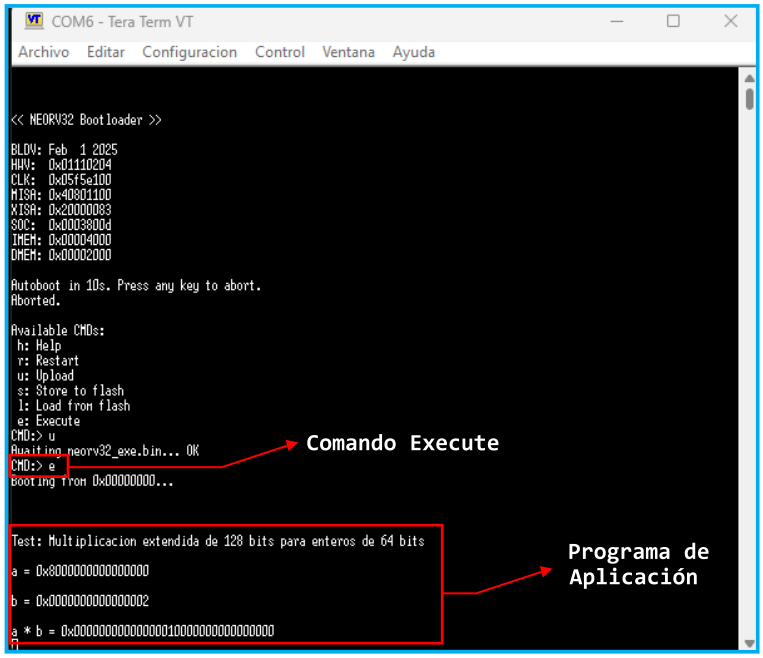

Fig.11. Ejecución del programa de aplicación mediante el comando *Execute*.

F.3. Carga Estática mediante Imagen VHDL.

Este método introduce el programa compilado dentro de la estructura de la FPGA, convirtiendo la memoria RAM de instrucciones típica de un procesador en una ROM preinicializada, en este caso con los datos binarios del programa compilado.

Para activar esta opción, el *generic* `BOOT_MODE_SELECT` debe ser igual a `2`, de esta forma el programa de inicio del procesador es directamente el de usuario.

F.3.1. Requisitos Previos.

Antes de comenzar el procedimiento de esta opción, es necesario haber llevado a cabo los siguientes apartados:

1. Haber realizado la creación de un proyecto nuevo en Vivado siguiendo la guía del Anexo E hasta el punto 18. En ese mismo punto es importante verificar que la entidad superior del proyecto añadida sea `neorv32_test_setup_approm.vhd`.

2. Una vez comprobado, continuar la guía del Anexo E solo hasta el punto 23.

F.3.2. Procedimiento.

1. Generar el archivo de imagen VHDL. Para ello acceder desde el directorio software del proyecto a la carpeta que aloje el programa de usuario, por ejemplo:

```
~$ cd neorv32/sw/example/test_isa_m/
```

2. Ejecutar la generación de la imagen VHDL mediante el siguiente comando de makefile:

```
~/neorv32/sw/example/test_isa_m$ make clean_all image

Memory utilization:

   text      data          bss          dec          hex      filename
   2840         0            0         2840          b18      main.elf
Compiling image generator...
Generating neorv32_application_image.vhd
```

   Nota: también podría haberse empleado el comando makefile: `make clean_all image install`, este comando sustituye al `neorv32_application_image.vhd` original alojado en `neorv32/rtl/core` por el recién generado. Una vez hecho esto, si se copian todos los VHDL del procesador directamente desde la carpeta `core` al proyecto de Vivado, no haría falta realizar los pasos 3, 4, 5 y 6.

3. Guardar el archivo `neorv32_application_image.vhd` generado en el sistema operativo donde se encuentre el programa Vivado.

4. Antes de añadir la imagen nueva al proyecto de Vivado, debemos eliminar la imagen original. Accedemos a la pestaña `Libraries` en la ventana `Sources`, hacemos clic derecho en el archivo `neorv32_application_image.vhd` y seleccionamos `Remove File from Project` (Fig.12).

5. Añadir la imagen VHDL generada al proyecto de Vivado. Para ello pulsar en `Add Sources`. En la ventana emergente seleccionamos `Add or create design sources` y pulsamos en `Next`.

6. En la ventana `Add or Create Design Sources` hacemos clic en `Add Files`, seleccionamos la imagen VHDL desde la dirección donde se haya almacenado y pulsamos en `OK` (Fig.13).

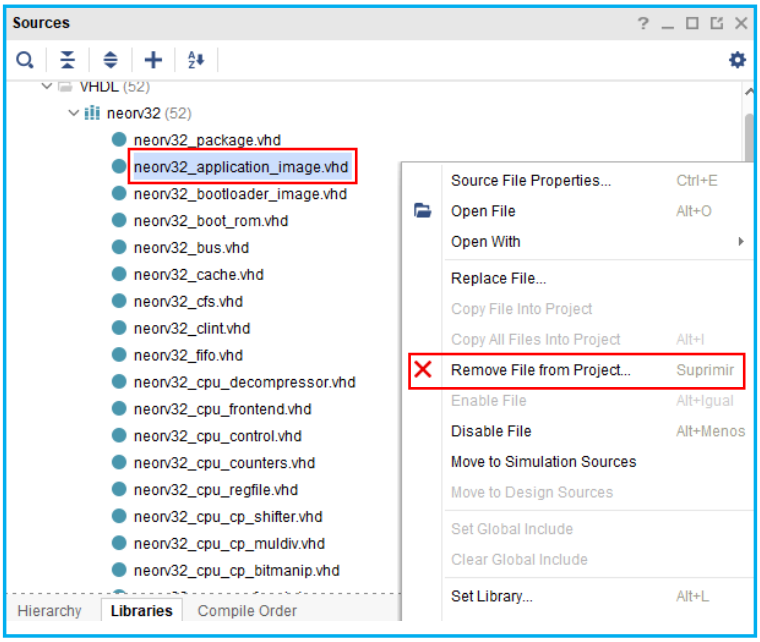

Fig.12. Borrado de la imagen VHDL del programa de aplicación original.

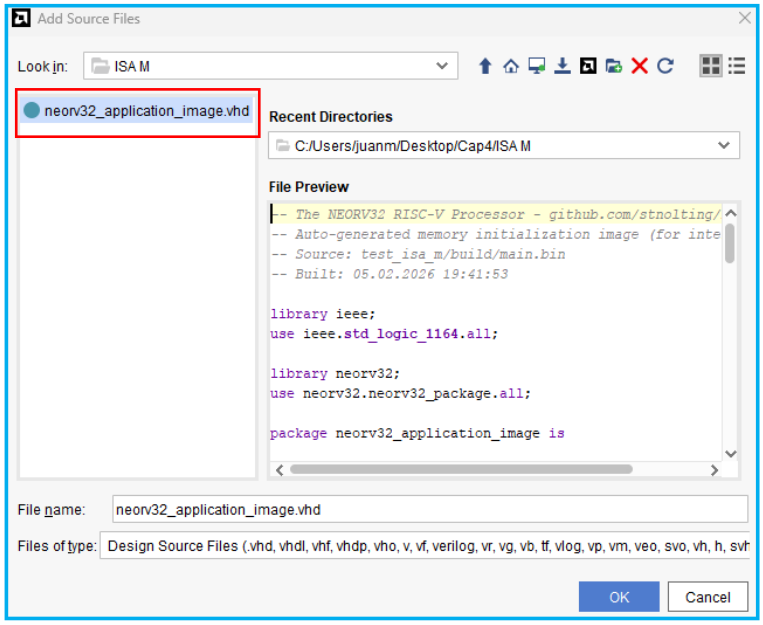

Fig.13. Carga de la nueva imagen VHDL del programa de aplicación.

7. De vuelta en la ventana anterior, nos aseguramos de que el campo `Library` del archivo sea neorv32 y que la opción `Copy sources into project` esté seleccionada (Fig.14). Pulsamos `Finish` para continuar.

8. El siguiente paso es opcional. En este ejemplo se usa un programa de aplicación que hace uso del puerto serie para mostrar datos de salida por el terminal. En la configuración de la entidad superior `neorv32_test_setup_approm.vhd` no viene

activada la UART por defecto. Además, el código de usuario hace uso de la extensión ISA M. Por estos motivos, se necesitan hacer las siguientes modificaciones en el archivo:

Listado 1. Modificaciones en el archivo neorv32_test_setup_approm.vhd para activar la unidad UART0 y la extensión ISA M.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

library neorv32;
use neorv32.neorv32_package.all;

entity neorv32_test_setup_approm is
  generic (
    -- adapt these for your setup --
    CLOCK_FREQUENCY   : natural := 100000000; -- clock frequency of clk_i in
Hz
    MEM_INT_IMEM_SIZE : natural := 16*1024;   -- size of processor-internal
instruction memory in bytes
    MEM_INT_DMEM_SIZE : natural := 8*1024     -- size of processor-internal
data memory in bytes
  );
  port (
    -- Global control --
    clk_i  : in  std_ulogic; -- global clock, rising edge
    rstn_i : in  std_ulogic; -- global reset, low-active, async
    -- GPIO --
    gpio_o : out std_ulogic_vector(7 downto 0); -- parallel output
    -- UART --
    uart0_txd_o    : out std_ulogic;  -- UART0 send data
    uart0_rxd_i    : in  std_logic    -- UART0 recive data
  );
end entity;

architecture neorv32_test_setup_approm_rtl of neorv32_test_setup_approm is

  signal con_gpio_out : std_ulogic_vector(31 downto 0);

begin

  -- The Core Of The Problem ----------------------------------------------
  -- -----------------------------------------------------------------------
  neorv32_top_inst: neorv32_top
  generic map (
    -- Clocking --
    CLOCK_FREQUENCY   => CLOCK_FREQUENCY,   -- clock frequency of clk_i in Hz
    -- Boot Configuration --
    BOOT_MODE_SELECT  => 2,                 -- boot from pre-initialized IMEM
    -- RISC-V CPU Extensions --
    RISCV_ISA_C       => false,           -- implement compressed extension?
    RISCV_ISA_M       => true,              -- implement mul/div extension?
    RISCV_ISA_Zicntr  => true,              -- implement base counters?
    -- Internal Instruction memory --
    MEM_INT_IMEM_EN   => true,              -- implement processor-internal
instruction memory
    MEM_INT_IMEM_SIZE => MEM_INT_IMEM_SIZE, -- size of processor-internal
instruction memory in bytes
    -- Internal Data memory --
    MEM_INT_DMEM_EN   => true,              -- implement processor-internal
data memory
```

```
    MEM_INT_DMEM_SIZE => MEM_INT_DMEM_SIZE, -- size of processor-internal data
memory in bytes
    -- Processor peripherals --
    IO_GPIO_NUM       => 8,       -- number of GPIO input/output pairs (0..32)
    IO_CLINT_EN       => true,                -- implement core local
interruptor (CLINT)?
    IO_UART0_EN       => true,                -- implement primary universal
asynchronous receiver/transmitter (UART0)?
    -- Tuning Options --
    CPU_FAST_MUL_EN   => true
  )
  port map (
    -- Global control --
    clk_i  => clk_i,       -- global clock, rising edge
    rstn_i => rstn_i,      -- global reset, low-active, async
    -- GPIO (available if IO_GPIO_NUM > 0) --
    gpio_o => con_gpio_out, -- parallel output
    -- UART --
    uart0_txd_o => uart0_txd_o,
    uart0_rxd_i => uart0_rxd_i
  );

  -- GPIO output --
  gpio_o <= con_gpio_out(7 downto 0);


end architecture;
```

9. Tras estas modificaciones, se puede retomar la guía del Anexo E desde el punto en el que se dejó. Las modificaciones del archivo de restricción de pines (.xdc) ahí mencionadas sirven también para este Anexo.

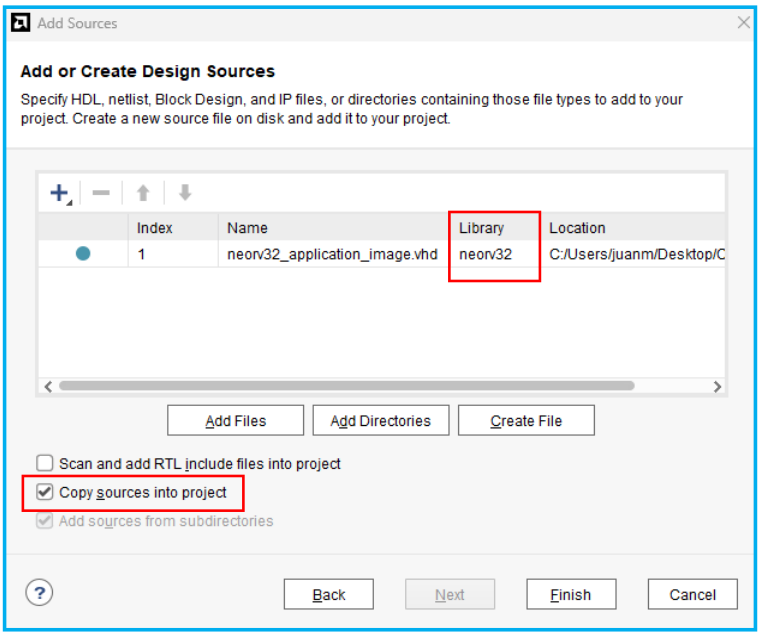

Fig.14. Ventana `Add or Create Design Sources`, tras añadir la imagen VHDL.

10. Con el *bitstream* generado y la FPGA programada, esta vez no tendremos una indicación de que todo ha ido bien (`LD0` parpadeando), dado que el programa del

bootloader ya no es lanzado. Por esta razón, necesitaremos crear una sesión en el puerto serie, tal y cómo se describió en el apartado F.2.2.

11. Con el terminal serie correctamente configurado, al activar el `SW0` (`rstn_i = '1'`, desactivado), el programa de aplicación de usuario se iniciará automáticamente mostrando los resultados por el terminal (Fig.15).

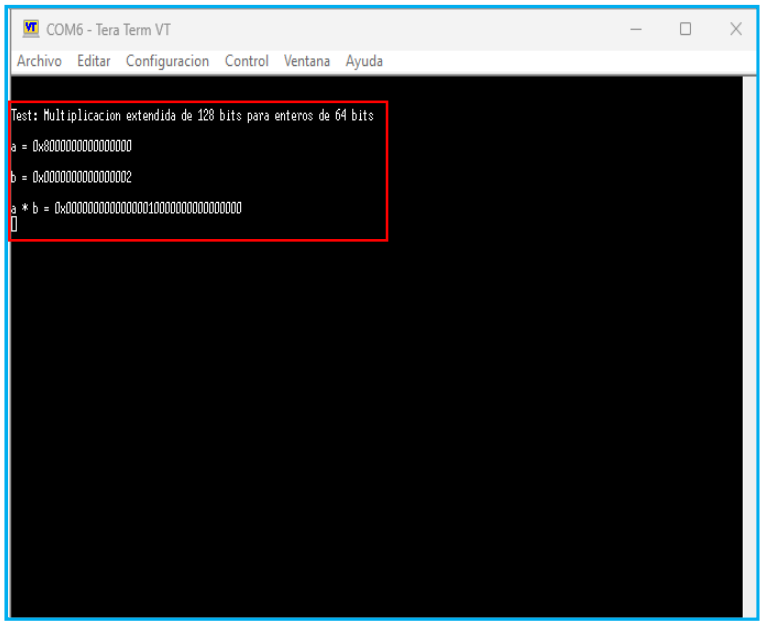

Fig.15. Captura del terminal del puerto serie, tras desactivar el *reset* del procesador, el programa de aplicación finalizó con éxito.

12. Dado que ahora el programa software se encuentra en memoria no volátil, cada vez que activemos el *reset* del procesador (`rstn_i = '0'`), al desactivarlo se volverá a ejecutar el programa (Fig.16).

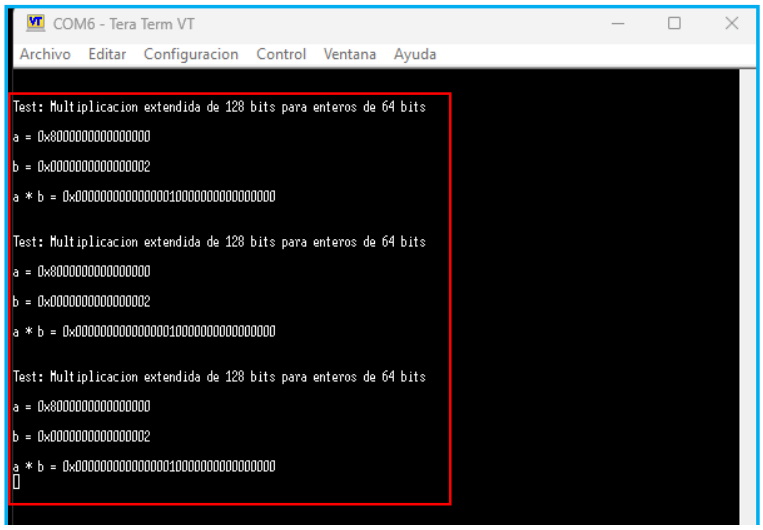

Fig.16. Captura del terminal del puerto serie tras tres ciclos de activación/desactivación del *reset*, el programa se ejecutó tres veces.